Retirement Quotes Automation — Complete Client Walkthrough Guide

Every Node, What It Does, Why It Exists

---

1. What The Client Asked For

The Business Problem

Aptia is a pension administration company. When a pension scheme member approaches retirement, they request a **Retirement Quote** — a document showing exactly how much pension they will receive, what lump sum they can take, and what their annuity rate is.

Currently this process is **fully manual**:

- An administrator receives the request in CAP (the pension CRM system)
- They manually check if the member is eligible
- They manually log into BeSt and create a case
- They manually run calculations in DocProd
- They manually generate and email the quote document
- They manually update the case at every step

This takes days, involves heavy human effort, and carries high risk of errors.

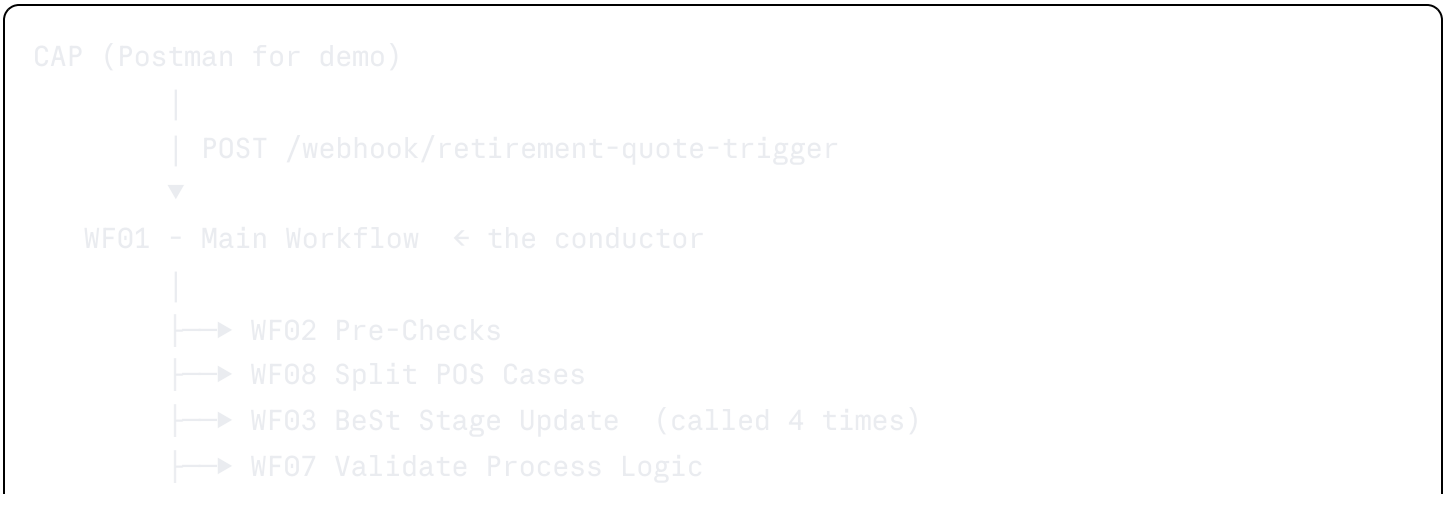
What The Client Wanted

> "Automate the end-to-end retirement quote process — from receiving the request to sending the quote to the member — with minimal human involvement."

Requirement	What It Means
Receive request from CAP	System triggers automatically when CAP sends a retirement quote request
Conduct DPA checks	Verify the member's identity before processing
Conduct pre-checks	Check age, status, AVC flag, POS count, ill health, vulnerable
Create a BeSt case	Register the case in the pension case management system
Run calculations / DocProd	Calculate pension figures and produce the quote document
Update BeSt at every stage	Keep case tracking system updated throughout
Send quote via email	Deliver the quote to member or their financial advisor (IFA)
Handle ineligible cases	Automatically close cases that cannot be processed
Handle AVC cases	Route cases with Additional Voluntary Contributions to a human
Handle multiple Periods of Service	Create separate cases for members with more than one employment period
Upload documents to Drive/SharePoint	Store all generated documents for audit and compliance
Update milestone questions	Keep member informed of case progress at each stage

2. What We Built

Architecture — 8 Workflows + Supporting Systems



```
        ├──► WF04 DocProd Integration
        ├──► WF06 Gmail Communication
        └──► WF05 One and Done Handler
```

Demo Replacements

Production System	Demo Replacement	Why
DocProd API (PDF)	Google Drive text file	Licensed Aptia system — not available in demo
SharePoint (storage)	Google Drive	Same upload pattern, different destination
RPA BOT (email)	Gmail node	Same outcome — quote delivered to member
CAP / BeSt APIs	Python mock server (127.0.0.1:5005)	Real APIs not accessible in demo
oPen API (calculations)	Hardcoded demo figures	Separate licensed calculation engine
Encrypted pack	Plain text attachment	DocProd handles encryption in production

3. Overall Process Flow

```
STEP 1: Request arrives from CAP via webhook
STEP 2: Scope + process type validation
STEP 3: Eligibility pre-checks (7 checks)
STEP 4: Route — PASS / ONE_AND_DONE / AVC_HOLD / ADMIN_CASE
STEP 5: Split POS if posCount > 1
STEP 6: BeSt VALIDATE_CASE stage update
STEP 7: Calculate retirement type, pack type, scheme rules
STEP 8: BeSt VALIDATE_PROCESS stage update
STEP 9: Generate documents, upload to Drive
STEP 10: BeSt PERFORM_PROCESS stage update + Ret Options
STEP 11: Email quote to member or IFA
STEP 12: BeSt ISSUE_DOCUMENTATION stage update
STEP 13: Push to Final Review queue
STEP 14: Log completion + respond to CAP
```

---

4. WF01 — Main Workflow (The Conductor)

Purpose: Receives the incoming request from CAP, validates it, and orchestrates all sub-workflows in the correct order. Every piece of data flows through this workflow.

Total nodes: 33

Node 1 — CAP Retirement Quote Trigger

Type: Webhook node **What it does:** Listens at the URL `POST /webhook/retirement-quote-trigger`. When CAP sends a POST request with member data, this node wakes up the entire workflow. **Why:** This is the entry point. In production, CAP automatically fires this when a retirement quote request is validated and ready for automation. For demo, Postman sends the request.

Node 2 — Map Payload Fields

Type: Set node **What it does:** Takes all the raw incoming data from the webhook body (`$json.body.*`) and maps it into clean, named fields that every subsequent workflow can use. Also adds `receivedAt` timestamp, and maps IFA fields, ill health flag, and vulnerable flag. **Why:** The webhook delivers data inside a `body` object. If we reference `$json.body.memberId` 50 times across 8 workflows it becomes error-prone. This node creates a clean flat structure once and all other nodes reference it.

Key fields mapped:

```
bestCaseId, memberId, schemeName, schemeCode, processType,
memberName, memberEmail, dateOfBirth, retirementDate,
membershipStatus, commPreference, posCount, hasAVC,
senderType, ifaEmail, ifaName, illHealth, vulnerableFlag
```

Node 3 — Scheme In Scope?

Type: IF node **What it does:** Checks if `schemeName` is in the approved list of schemes we automate: EGP, Cadent, JLR, Smiths, Kelda, Kingfisher, BAES, PCS. **Why:** Not all pension schemes are set up for automation. Schemes outside this list have different rules, templates, or are not yet configured. We must stop before doing anything that could create incorrect data.

- **True (in scope)** → continues to Process check

- **False (not in scope)** → STOP - Not In Scope

Node 4 — STOP - Not In Scope

Type: Set node **What it does:** Returns a stopped status with reason "Scheme not in scope for retirement quote automation." **Why:** Provides a clear, traceable output that CAP can read to understand why the request was not processed. No case is created, no BeSt updates happen.

Node 5 — Process = Retirement Quote?

Type: IF node **What it does:** Checks if `processType === "Retirement Quote"`. **Why:** The same webhook could receive different types of requests (Transfer Out, Death Benefit, etc). Each process type has different automation logic. This workflow only handles Retirement Quotes. Any other type must stop here.

- **True** → continues to pre-checks
- **False** → STOP - Wrong Process

Node 6 — STOP - Wrong Process

Type: Set node **What it does:** Returns stopped status with reason "Process type is not Retirement Quote." **Why:** Same as above — clean, traceable output for CAP.

Node 7 — Run Pre-Checks

Type: Execute Workflow node **What it does:** Calls WF02 (Pre-Checks Sub-Workflow) and passes all member data to it. Waits for WF02 to complete and return a result. **Why:** Eligibility checking is complex (7 checks) and is best kept in its own workflow for clarity and reusability. WF01 delegates this responsibility entirely to WF02.

Node 8 — Result = PASS?

Type: IF node **What it does:** Checks if `preCheckResult === "PASS"` returned from WF02. **Why:** All 7 pre-checks passed — member is eligible for automated processing. This is the gate that separates the happy path from all exception paths.

- **True (PASS)** → AVC check

- **False** → check for ONE_AND_DONE

Node 9 — If (AVC Flagged?)

Type: IF node **What it does:** Checks if `avcFlagged === true`. **Why:** Members with AVC (Additional Voluntary Contributions) require a human to verify the current AVC value before a quote can be produced. The automation cannot safely pull this figure automatically — it changes frequently and must be manually confirmed.

- **True (AVC exists)** → Send admin email → Wait for form
- **False (no AVC)** → Split POS Cases (continue automation)

Node 10 — Send a message (AVC Admin Email)

Type: Gmail node **What it does:** Sends an email to the admin team with a unique form link (`$execution.resumeFormUrl`). The email explains which case needs AVC verification and what the admin needs to do. **Why:** The admin team need to be notified that a case is paused and waiting for them. The form link is unique to this execution — it's the mechanism for the admin to resume the workflow after inputting the AVC value.

Node 11 — Wait (AVC Form)

Type: Wait node (form mode) **What it does:** Pauses the entire workflow and displays a form when the admin clicks the link from the email. The form shows case details and asks the admin to enter: AVC value (£), AVC provider name, and any notes. When the admin submits, the workflow immediately resumes. **Why:** This is the human-in-the-loop mechanism. The workflow cannot proceed without the AVC value — it would produce an incorrect quote. The Wait node is the only n8n mechanism that can pause a workflow indefinitely until a human responds.

Node 12 — AVC Hold Set node

Type: Set node **What it does:** After the form is submitted, this node captures the form response (`avcValue`, `avcProvider`, `avcAdminNotes`) and rebuilds the full member payload by referencing back to `Map Payload Fields`. Sets `avcConfirmed: true`. **Why:** When the Wait node resumes, `$json` only contains the form data — all original member fields (bestCaseId, memberId, etc) are gone. This node merges both: form data + original member data = complete payload to continue.

Node 13 — Result = ONE AND DONE?

Type: IF node **What it does:** Checks if `preCheckResult === "ONE_AND_DONE"`. **Why:** Multiple pre-check failures result in ONE_AND_DONE — the case must be closed automatically with an explanation to the member. This routes those cases to WF05.

- **True** → One and Done Handler (WF05)
- **False** → check for ADMIN_CASE

Node 14 — One and Done Handler

Type: Execute Workflow node **What it does:** Calls WF05 which handles case closure, member notification, and BeSt case closing. **Why:** Closing a case requires multiple steps (build message, call mock server, send email, close in BeSt). Delegated to WF05.

Node 15 — Result = ADMIN CASE?

Type: IF node **What it does:** Checks if `preCheckResult === "ADMIN_CASE"`. **Why:** Some membership statuses (Unknown, Other) can't be processed automatically but also don't warrant a ONE_AND_DONE closure. They need a human to review. This routes them to the admin team.

- **True** → Create Admin Team Case
- **False** → STOP - Unknown Result

Node 16 — Create Admin Team Case

Type: HTTP Request node **What it does:** POSTs to `http://127.0.0.1:5005/api/cases/admin-review` with member details. Creates an admin review record in the mock server. **Why:** The admin team need a formal case record to work from. In production this creates a work item in the admin queue in CAP. For demo, the mock server stores it.

Node 17 — STOP - Unknown Result

Type: Set node **What it does:** Catches any unrecognised `preCheckResult` values and returns an error status. **Why:** Defensive programming. Should never happen in practice but if WF02 returns something unexpected, this prevents the workflow from silently failing or routing incorrectly.

Node 18 — Split POS Cases

Type: Execute Workflow node **What it does:** Calls WF08 to check if member has multiple periods of service and split if needed. **Why:** Members who worked in the scheme at different times (e.g. left and rejoined) have multiple Periods of Service. Each POS needs its own BeSt case. WF08 handles this complexity.

Node 19 — Set Stage - VALIDATE_CASE

Type: Code node **What it does:** Adds `stageName: "VALIDATE_CASE"` to the data. This tells WF03 which stage to update in BeSt. **Why:** WF03 is reused 4 times. The only difference each time is which stage to update. This code node injects the correct stage name before calling WF03. Uses a code node (not Set) to spread all existing fields cleanly.

Node 20 — BeSt - Validate Case

Type: Execute Workflow node **What it does:** Calls WF03 with `stageName: VALIDATE_CASE`. WF03 verifies the case exists in BeSt and updates the stage. **Why:** BeSt must be updated at each process milestone for compliance and case tracking. VALIDATE_CASE is the first milestone — confirms the case is registered and validated.

Node 21 — Validate Process Logic

Type: Execute Workflow node **What it does:** Calls WF07 to calculate retirement type (Early/Normal/Late), pack type (FullPack/EstimateLetter), and check scheme rules (early retirement allowed? trustee approval needed? consent required?). **Why:** These calculations require Supabase lookups and business logic. Delegated to WF07 to keep WF01 clean.

Node 22 — Set Stage - VALIDATE_PROCESS

Type: Code node **What it does:** Adds `stageName: "VALIDATE_PROCESS"` to data. **Why:** Same pattern as VALIDATE_CASE — injects correct stage name before calling WF03 again.

Node 23 — BeSt - Validate Process

Type: Execute Workflow node **What it does:** Calls WF03 with `stageName: VALIDATE_PROCESS`. **Why:** Second BeSt milestone — confirms the process logic has been validated (retirement type confirmed, rules checked).

Node 24 — DocProd Integration

Type: Execute Workflow node **What it does:** Calls WF04 to generate the retirement quote document and upload it to Google Drive. Returns `driveFileId` and `fileName`. **Why:** Document production involves multiple steps (job state check, Supabase config lookup, letter building, folder management, file uploads). Delegated to WF04.

Node 25 — DocProd Success?

Type: IF node **What it does:** Checks if WF04 returned `status === "SUCCESS"`. **Why:** Document generation can fail (job not ready, Drive API error, etc). If it fails, the workflow must not continue to email — we'd be sending a broken or missing attachment.

- **True** → continues to PERFORM_PROCESS stage
- **False** → Flag Manual Review

Node 26 — Flag Manual Review

Type: Set node **What it does:** Returns `workflowStatus: "MANUAL_REVIEW_REQUIRED"` with the error from WF04. **Why:** Document generation failure requires human intervention. The case must not be silently abandoned — it's flagged so an admin can pick it up and investigate.

Node 27 — Set Stage - PERFORM_PROCESS

Type: Code node **What it does:** Adds `stageName: "PERFORM_PROCESS"` to data. **Why:** Third BeSt milestone injection before WF03 call.

Node 28 — BeSt - Perform Process

Type: Execute Workflow node **What it does:** Calls WF03 with `stageName: PERFORM_PROCESS`. WF03 also triggers a Ret Options update at this stage. **Why:** PERFORM_PROCESS is the stage where the document has been produced. BeSt records that the process has been performed and the Retirement Options section is updated with retirementType and packType.

Node 29 — RPA Communication

Type: Execute Workflow node **What it does:** Calls WF06 to download the letter from Drive and email it to the member or IFA. **Why:** Email communication involves Drive download, IFA routing logic, Gmail send, CAP update, and Final Review push. Delegated to WF06 which replaces the RPA BOT.

Node 30 — Set Stage - ISSUE_DOCUMENTATION

Type: Code node **What it does:** Adds `stageName: "ISSUE_DOCUMENTATION"` to data. **Why:** Fourth and final BeSt milestone injection.

Node 31 — BeSt - Issue Documentation

Type: Execute Workflow node **What it does:** Calls WF03 with `stageName: ISSUE_DOCUMENTATION`. **Why:** Final BeSt milestone — confirms the quote has been issued to the member. Completes the case tracking lifecycle.

Node 32 — Log Completion

Type: Set node **What it does:** Sets `workflowStatus: "COMPLETED"` with timestamp and all key case fields. **Why:** Creates a clean final output record. CAP reads this to confirm the automation completed successfully. Also useful for execution logs and auditing.

Node 33 — Respond to CAP WebHook

Type: Respond to Webhook node **What it does:** Sends the final response back to CAP with the completion status and all case details. **Why:** The webhook was opened at Node 1 and has been waiting for a response. Without this node, the HTTP connection would time out. This formally closes the request-response cycle and confirms to CAP that automation is complete.

---

5. WF02 — Pre-Checks Sub-Workflow

Purpose: Runs 7 sequential eligibility checks on the member. Returns PASS, ONE_AND_DONE, or ADMIN_CASE to WF01.

Total nodes: 30

Node 1 — Pre-Checks Entry

Type: Execute Workflow Trigger **What it does:** Receives all member data from WF01 when called. This is the starting point of WF02. **Why:** Every sub-workflow starts with this trigger node. It receives the input data passed from the Execute Workflow node in WF01 and makes it available as `$json`.

Node 2 — Calculate Age

Type: Code node **What it does:** Calculates the member's age at their requested retirement date. Takes `dateOfBirth` and `retirementDate`, computes years between them, and adds `ageAtRetirement` to the data. Also spreads all original payload fields so nothing is lost. **Why:** Age eligibility is the first and most fundamental check. The calculation must be precise — it's not just current age but age at the specific retirement date requested. Using a code node allows the mathematical calculation that a Set node cannot do.

Node 3 — CHECK 1 - Age OK?

Type: IF node **What it does:** Checks if `ageAtRetirement >= 55`. **Why:** Pension regulations require a minimum retirement age of 55. Quoting for someone below this age is not just wrong — it could create regulatory issues. This check is non-negotiable.

- **True (≥55)** → DPA check
- **False (<55)** → RESULT - Under 55

Node 4 — RESULT - Under 55

Type: Set node **What it does:** Returns `preCheckResult: "ONE_AND_DONE"` with `preCheckReason: "UNDER_AGE_55"` and a member-friendly message. **Why:** Case cannot proceed. WF01 routes this to WF05 which will close the case and notify the member that they do not yet meet the minimum retirement age.

Node 5 — CHECK 2 - DPA Check (Supabase)

Type: Supabase node **What it does:** Queries the `dpa_checks` table in Supabase using `member_id = memberId`. Looks for a row where `dpa_result = "Pass"`. **Why:** Data Protection Act (DPA) checks verify the member's identity before any personal pension data is processed or shared. This is a legal requirement. Without DPA verification, processing the case would be a data

protection violation. Supabase stores these records as they are managed by the compliance team independently.

Node 6 — DPA Pass?

Type: IF node **What it does:** Checks if the Supabase query returned a row with `dpa_result = "Pass"`. **Why:** If the member is not in the DPA table or their result is not Pass, they have not been identity verified. Processing must stop immediately.

- **True (DPA passed)** → status check
- **False (DPA failed or not found)** → RESULT - DPA Failed

Node 7 — RESULT - DPA Failed

Type: Set node **What it does:** Returns `preCheckResult: "ONE_AND_DONE"` with `preCheckReason: "DPA_FAILED"`. **Why:** Case cannot proceed without identity verification. WF05 will close it and the member will be directed to contact the admin team to resolve their DPA status.

Node 8 — CHECK 3 - Get Status (Code)

Type: Code node **What it does:** Extracts the `membershipStatus` from the payload (spreads all fields, ensures status is available for the IF chain below). **Why:** After the Supabase node, `$json` contains the Supabase row data — the original payload fields are replaced. This code node restores them by reading from `$('Calculate Age').item.json` and spreading everything.

Node 9 — Status = Active?

Type: IF node **What it does:** Checks if `membershipStatus === "Active"`. **Why:** Active members are current contributing members — fully eligible for a retirement quote.

- **True** → Status OK - Active
- **False** → check Deferred

Node 10 — Status = Deferred?

Type: IF node **What it does:** Checks if `membershipStatus === "Deferred"`. **Why:** Deferred

members left the scheme but their pension remains in it — they are also eligible for a retirement quote.

- **True** → Status OK - Deferred
- **False** → check No Liability

Node 11 — Status = No Liability?

Type: IF node **What it does:** Checks if `membershipStatus === "No Liability"`. **Why:** No Liability means the scheme has no pension obligation to this member (e.g. benefits transferred out). There is nothing to quote on.

- **True** → RESULT - No Liability
- **False** → check Pensioner

Node 12 — Status = Pensioner?

Type: IF node **What it does:** Checks if `membershipStatus === "Pensioner"`. **Why:** Pensioners are already receiving their pension. They cannot retire again — there is nothing to quote on.

- **True** → RESULT - Pensioner
- **False** → RESULT - Other Status

Nodes 13-17 — Status OK - Active / Status OK - Deferred / RESULT - No Liability / RESULT - Pensioner / RESULT - Other Status

Type: Set nodes **What they do:** Each outputs the appropriate `preCheckResult` with all member fields:

- `Status OK - Active/Deferred` → sets `statusOK: true` and continues to POS check
- `RESULT - No Liability` → ONE_AND_DONE
- `RESULT - Pensioner` → ONE_AND_DONE
- `RESULT - Other Status` → ADMIN_CASE (unknown status needs human review)

Node 18 — CHECK 4 - Get POS (Code)

Type: Code node **What it does:** Reads `posCount` and `posDetails` from the payload. Spreads all

fields for the next check. **Why:** POS count determines if WF08 needs to split the case. This node ensures the count is available and normalised. Also uses a code node because `$json` after Status OK nodes needs careful field spreading.

Node 19 — Multiple POS?1

Type: IF node **What it does:** Checks if `posCount > 1`. **Why:** Multiple POS cases are handled differently — WF08 creates one BeSt case per POS. This flag must be captured before RESULT - PASS so WF01 knows to call WF08 with the correct data.

- **True** → Flag Multi POS → RESULT - PASS (with flag set)
- **False** → CHECK 5 AVC

Node 20 — Flag Multi POS1

Type: Set node **What it does:** Sets `preCheckResult: "MULTI_POS"` to flag the condition, then this feeds into RESULT - PASS. Includes all POS details. **Why:** WF01 still proceeds with the happy path but now knows WF08 needs to split the case.

Node 21 — CHECK 5 - Get AVC (Code)1

Type: Code node **What it does:** Reads `hasAVC` from payload. Spreads all fields. **Why:** AVC flag determines if the automation needs to pause for human input. Code node ensures clean field handling.

Node 22 — AVC Ticked?1

Type: IF node **What it does:** Checks if `hasAVC === true`. **Why:** AVC cases need human verification of the AVC value. They don't fail — they pause. Both paths eventually reach RESULT - PASS with `avcFlagged` set appropriately.

- **True** → Log AVC → CHECK 6 Ill Health
- **False** → CHECK 6 Ill Health

Node 23 — Log AVC1

Type: Set node **What it does:** Sets `avcFlagged: true` and adds `avcNote` explaining the flag.

Passes all member fields through. **Why:** Records that AVC was detected. WF01 uses `avcFlagged: true` to trigger the Gmail + Wait human in loop process.

Node 24 — CHECK 6 - Get Ill Health

Type: Code node **What it does:** Reads `illHealth` from payload. Spreads all fields. **Why:** Ill health retirement is a separate process — it cannot be automated. Checking it here ensures it's caught before any processing happens.

Node 25 — Ill Health Request?

Type: IF node **What it does:** Checks if `illHealth === true`. **Why:** Ill health retirement requires specialist actuarial review. Automating it without specialist input could result in incorrect benefit calculations with serious financial and legal consequences.

- **True** → RESULT - Ill Health (ONE_AND_DONE)
- **False** → Vulnerable Flag check

Node 26 — RESULT - Ill Health

Type: Set node **What it does:** Returns ONE_AND_DONE with reason `ILL_HEALTH_REQUEST`. **Why:** Routes to WF05 which closes the case and informs the member it needs specialist handling.

Node 27 — Vulnerable Flag

Type: Code node **What it does:** Reads `vulnerableFlag` from payload. Spreads all fields. **Why:** Vulnerable members need extra care — their cases need manual checker review at the end. Does NOT stop automation.

Node 28 — Vulnerable Member?

Type: IF node **What it does:** Checks if `vulnerableFlag === true`. **Why:** Vulnerable members still get their quote automatically but the checker is alerted. The Final Review stage provides the human oversight needed.

- **True** → Log Vulnerable → RESULT - PASS (with flag)
- **False** → RESULT - PASS directly

Node 29 — Log Vulnerable

Type: Set node **What it does:** Sets `vulnerableFlag: true` and adds `vulnerableNote` explaining that manual review is required at checkout. All member fields passed through. **Why:** Ensures WF01 and downstream workflows carry the vulnerable flag so the checker at Final Review is aware.

Node 30 — RESULT - PASS1

Type: Set node **What it does:** Final output node. Sets `preCheckResult: "PASS"` and `canProceed: true`. Carries all member fields including `avcFlagged`, `vulnerableFlag`, `illHealth`, `posCount`, `posDetails`, `ageAtRetirement`, `checkedAt` timestamp. **Why:** This is what WF01 receives. All 7 checks have passed. The member is eligible. WF01 reads `preCheckResult === "PASS"` to continue the automation.

---

6. WF03 — BeSt Stage Update Sub-Workflow

Purpose: Updates the BeSt case at each of 4 process stages. Called 4 times from WF01 with different `stageName` each time. Also updates Ret Options (at PERFORM_PROCESS) and milestone questions (at every stage).

Total nodes: 12

Node 1 — BeSt Update Entry

Type: Execute Workflow Trigger **What it does:** Receives all member data plus `stageName` from WF01. **Why:** Entry point for WF03. The `stageName` field tells WF03 which stage to update.

Node 2 — GET BeSt Case

Type: HTTP Request node **What it does:** `GET http://127.0.0.1:5005/api/cases/{bestCaseId}` — verifies the case exists in the mock server before attempting to update it. **Why:** You cannot update a case that doesn't exist. If BeSt doesn't have this case (maybe it was never created, or wrong ID) the update would fail silently or create corrupt data. This GET confirms existence first.

Node 3 — Case Found?

Type: IF node **What it does:** Checks if the GET response returned `ok: true` or a valid `caseId`. **Why:** Guards against attempting a stage update on a non-existent case.

- **True** → PUT Update Stage
- **False** → ERROR - Case Not Found

Node 4 — ERROR - Case Not Found

Type: Set node **What it does:** Returns `bestStageUpdated: false` with error message "BeSt case not found." **Why:** WF01 receives this and can log the failure. Case tracking shows the stage was not updated.

Node 5 — PUT Update Stage

Type: HTTP Request node **What it does:** `PUT http://127.0.0.1:5005/api/cases/{bestCaseId}/stage` with body `{ stageName, updatedAt, targetDate }`. Updates the case stage in BeSt. **Why:** This is the core purpose of WF03 — keeping BeSt in sync with where the automation is in the process. The `targetDate` (14 days from now) is added per the requirements to reduce overall TAT.

Node 6 — Update OK?

Type: IF node **What it does:** Checks if the PUT response returned `ok: true` or `updateStatus: "SUCCESS"`. **Why:** The PUT could fail (server error, case locked, etc). Must confirm success before recording it.

- **True** → Stage Updated OK
- **False** → ERROR - Update Failed

Node 7 — ERROR - Update Failed

Type: Set node **What it does:** Returns `bestStageUpdated: false` with error "Stage update failed." **Why:** Failure to update BeSt is a compliance issue — case tracking would be wrong. This error allows WF01 to flag it for investigation.

Node 8 — Stage Updated OK

Type: Set node **What it does:** Sets `bestStageUpdated: true` with the `stageName` and timestamp. Passes all member fields through referencing `BeSt Update Entry`. **Why:** Confirms stage was updated. All member fields must be re-output here because the HTTP response replaced `$json` — without this, WF01 would lose all member data.

Node 9 — Is PERFORM_PROCESS Stage?

Type: IF node **What it does:** Checks if `stageName === "PERFORM_PROCESS"`. **Why:** Ret Options section in BeSt only exists at the PERFORM_PROCESS stage. Calling the Ret Options endpoint at other stages would be wrong and could cause errors in the real system. This ensures Ret Options update only happens at the right moment.

- **True** → PUT Update Ret Options
- **False** → Edit Fields (terminal node)

Node 10 — PUT Update Ret Options

Type: HTTP Request node **What it does:** `PUT http://127.0.0.1:5005/api/cases/{bestCaseId}/ret-options` with `{ retirementType, packType, updatedAt }`. **Why:** Requirements explicitly state the Retirement Options section must be populated during Perform Process. This section records what type of retirement (Early/Normal/Late) and what pack type (FullPack/EstimateLetter) was used — critical for audit and future reference.

Node 11 — Ret Options Updated

Type: Set node **What it does:** Sets `retOptionsUpdated: true` flag. **Why:** Records that Ret Options update was done. Both PERFORM_PROCESS path (via this node) and other stage paths merge into Edit Fields — this flag distinguishes which path was taken.

Node 12 — Edit Fields

Type: Set node **What it does:** Final terminal node for ALL paths in WF03. Returns complete envelope: `status: "SUCCESS"`, all member fields from BeSt Update Entry, `bestStageUpdated`, `stageName`, `retOptionsUpdated`, `driveFileId`, `fileName`, `senderType`, `ifaEmail`, `ifaName`. **Why:** WF01 needs a consistent output from WF03 regardless of which stage was updated and

whether Ret Options ran. All member fields must be present so WF01 can pass them to the next sub-workflow.

---

7. WF04 — DocProd Integration Sub-Workflow

Purpose: Generates the retirement quote letter, calc record, and CAP checklist. Uploads all 3 to Google Drive. Returns `driveFileId` for WF06 to download and email.

Total nodes: 20

Node 1 — DocProd Entry

Type: Execute Workflow Trigger **What it does:** Receives all member data from WF01 including `retirementType`, `packType`, and `schemeName`. **Why:** Entry point. All letter content depends on these fields.

Node 2 — GET JobState Check

Type: HTTP Request node **What it does:** `GET http://127.0.0.1:5005/api/jobstate/{memberId}` — checks if the case is in a valid state for document generation. **Why:** In real BeSt, before DocProd can run calculations, the case must be in a "READY_FOR_CALCULATION" job state. If the case is locked, missing required fields, or not yet at the right stage, DocProd would fail or produce wrong results. This check prevents that. For demo the mock always returns `jobReady: true`.

Node 3 — Job Ready?

Type: IF node **What it does:** Checks `jobReady === true` from the JobState response. **Why:** Gate before document production. If job is not ready, no document should be generated.

- **True** → GET Scheme Config
- **False** → ERROR - Job Not Ready

Node 4 — ERROR - Job Not Ready

Type: Set node **What it does:** Returns `status: "ERROR"` with `docProdStatus: "ERROR"` and error message. **Why:** WF01 checks DocProd success — this failure stops the email from being sent with a missing or wrong document.

Node 5 — GET Scheme Config from Supabase

Type: Supabase node **What it does:** Queries `dss_scheme_config` table for the row matching `scheme_name = schemeName`. Returns pack type, document panel details, template settings. **Why:** Different schemes have different document templates, panel configurations, and retirement options ordering. The letter content must be scheme-specific.

Node 6 — Build Letter Content

Type: Code node **What it does:** Assembles the full retirement quote letter as a text string. Includes member details, scheme name, retirement date, retirement type, pack type, pension figures (£5,420/year), annuity rate (4.25%), lump sum (£16,260). Constructs `letterContent` and `fileName`. **Why:** This is the document production step. In production, DocProd generates a formatted PDF from actual calculated figures. For demo, we produce a text file with realistic dummy figures. Same flow, same file ID returned, same Drive storage — only the format and real calculation differ.

Node 7 — Simulate DocProd Response

Type: Set node **What it does:** Adds `continueToMember: true` to the data. **Why:** In real DocProd, the API response includes a `continueToMember` flag. If the calculation results in zero or negative values, DocProd sets this to `false` — meaning "do not send to member, human must fix the figures." For demo it's hardcoded `true` but the check is fully wired and working.

Node 8 — Continue To Member?

Type: IF node **What it does:** Checks `continueToMember === true`. **Why:** If false, the quote has bad figures. Sending it to the member would be a serious error — they'd receive incorrect pension information. This gate prevents that.

- **True** → Search Member Folder
- **False** → FLAG - Manual Review Required

Node 9 — FLAG - Manual Review Required

Type: Set node **What it does:** Returns `docProdStatus: "MANUAL_REVIEW"` with reason. **Why:** Human must correct the calculation figures before the quote can be issued.

Node 10 — Search Member Folder

Type: Google Drive node **What it does:** Searches Google Drive for an existing folder named `{memberId}` inside the `Retirement-Quotes` parent folder. **Why:** We organise files by member — each member has their own folder. Searching first prevents creating duplicate folders on repeated runs. Reuses existing folder if it's there.

Node 11 — Folder Exists?

Type: IF node **What it does:** Checks if Drive search returned any results. **Why:** Determines whether to upload to existing folder or create new one first.

- **True** → Upload Letter (Existing Folder)
- **False** → Create Member Folder

Node 12 — Create Member Folder

Type: Google Drive node **What it does:** Creates a new folder `{memberId}` inside `Retirement-Quotes` in Drive. **Why:** First time this member's case is processed — their folder doesn't exist yet. Must create before uploading.

Node 13 — Upload Letter (Existing/New Folder)

Type: Google Drive node (createFromText) **What it does:** Creates and uploads the retirement quote letter as a `.txt` file to the member's Drive folder. Returns `id` (the Drive file ID) and `name`. **Why:** This is the quote document that will be downloaded by WF06 and emailed to the member. The `id` is returned as `driveFileId` to WF01 which passes it to WF06.

Node 14 — Create Checklist (Existing/New)

Type: Google Drive node (createFromText) **What it does:** Creates and uploads `CAP_Checklist_{bestCaseId}.txt` to the member's Drive folder. Content is a pre-filled checklist of all steps that need checker sign-off. **Why:** Requirements state CAP checklist must be automatically uploaded to the case. The checker accesses this at Final Review stage — they open the Drive folder, work through the checklist, and sign off before the case is officially closed.

Node 15 — Create file from text (Calc Record)

Type: Google Drive node (createFromText) **What it does:** Creates and uploads `CalcRecord_{bestCaseId}.txt` with raw calculation figures: pension amount, annuity rate, lump sum, calc date, retirement type, pack type. **Why:** Requirements state the XML calc file must be uploaded (replacing SharePoint with Drive). This is the audit record of what figures were used in the quote. Internal only — never sent to the member. Replaces the DocProd XML output with a text equivalent for demo.

Node 16 — DocProd Complete (Existing/New)

Type: Set node **What it does:** Final output node. Returns complete envelope with:

- `status: "SUCCESS"`
- `driveFileId` — quote letter file ID (from Upload Letter node)
- `fileName` — quote letter file name
- `calcFileId` — calc record file ID
- `checklistFileId` — checklist file ID
- All member fields from DocProd Entry
- `continueToMember: true` **Why:** WF01 needs the `driveFileId` to pass to WF06 for email attachment. All member fields must be re-output because the Drive nodes replaced `$json`. Two separate Complete nodes exist (one per folder path) to reference the correct Drive node names.

---

8. WF05 — One and Done Handler Sub-Workflow

Purpose: Handles all cases that cannot proceed with automation. Closes the case, notifies the member, and updates BeSt.

Total nodes: 15

Node 1 — One and Done Entry

Type: Execute Workflow Trigger **What it does:** Receives member data and `preCheckReason` from WF01. **Why:** Entry point. The reason determines which message to build.

Nodes 2-5 — Reason = Under 55? / No Liability? / Pensioner? / DPA Failed?

Type: IF nodes (chain) **What they do:** Sequential checks of `preCheckReason` to identify which closure message to build. **Why:** Each reason has a different, appropriate message for the member. A chain of IF nodes routes to the correct message builder.

Nodes 6-10 — MSG Under 55 / MSG No Liability / MSG Pensioner / MSG DPA Failed / MSG Admin Case

Type: Set nodes **What they do:** Each builds a `messageText` appropriate to the closure reason:

- Under 55: "We are unable to process as your chosen date is before the minimum retirement age."
- No Liability: "We have confirmed you do not have any active benefits in this scheme."
- Pensioner: "Our records show you are already receiving a pension from this scheme."
- DPA Failed: "We are unable to process your request — case referred to admin team."
- Admin Case: "Your case has been referred to our administration team for review." **Why:** Generic messages are not acceptable — members must receive an explanation that makes sense for their specific situation. Tailored messages are a requirement.

Node 11 — Merge - Ready to Close

Type: Set node **What it does:** All 5 message paths converge here. Reads `messageText` from whichever message node ran and assembles the final closure payload with all member fields. **Why:** After the IF chain, all paths must produce the same output structure for the following HTTP calls and Gmail. This merge point normalises them.

Node 12 — Complete Case in CAP

Type: HTTP Request node **What it does:** `POST http://127.0.0.1:5005/api/cases/{bestCaseId}/complete` with `{ reason, closedAt }`. Marks the case as CLOSED in the mock server. **Why:** CAP must be told the case is closed so it doesn't keep it in an open/pending state. In production, this updates the CAP work item status.

Node 13 — Send Message to Member

Type: Gmail node **What it does:** Sends a notification email to the member's email address explaining why their case was closed. **Why:** Members must be informed when their retirement

quote request cannot be processed automatically. Leaving them with no response would be poor service and a compliance issue.

Node 14 — BeSt - Close Case

Type: HTTP Request node **What it does:** `PUT http://127.0.0.1:5005/api/cases/{bestCaseId}/stage` with `stageName: "CLOSED"`. Marks the case as closed in BeSt. **Why:** BeSt must reflect the true state of every case. A case that was auto-closed must be marked CLOSED in BeSt — not left in VALIDATE_CASE or whatever stage it was at when stopped.

Node 15 — Log Closed

Type: Set node **What it does:** Sets `oneAndDoneComplete: true` with timestamp and all case fields. **Why:** Final output back to WF01. Confirms closure is complete.

---

9. WF06 — Gmail Communication Sub-Workflow

Purpose: Downloads the quote letter from Drive and emails it to the member or IFA. Replaces the RPA BOT.

Total nodes: 14

Node 1 — RPA Entry

Type: Execute Workflow Trigger **What it does:** Receives all member data including `driveFileId` from WF04. **Why:** Entry point. `driveFileId` is the key piece of data needed to download the letter.

Node 2 — Prepare Payload

Type: Set node **What it does:** Reorganises fields into clean names for use in this workflow: `documentFileId`, `memberEmail`, `memberId`, `commType`, `fileName`, `schemeName`, `bestCaseId`, `memberName`, `retirementType`, `packType`. **Why:** After the Execute Workflow Trigger, field names are exactly as received. This node renames and organises them for clean referencing throughout WF06.

Node 3 — Recipient = IFA?

Type: IF node **What it does:** Checks if `senderType === "IFA"`. **Why:** Requirements state the quote must go to the IFA (financial advisor) when one is managing the member's case — not directly to the member. The `senderType` field in the payload controls this routing.

- **True** → Set IFA as Recipient
- **False** → Set Member as Recipient

Node 4 — Set IFA as Recipient

Type: Set node **What it does:** Sets `recipientEmail = ifaEmail` and `recipientName = ifaName`. **Why:** The Gmail node will use these fields — keeping them generic (not hardcoded to memberEmail) makes the IFA routing work cleanly.

Node 5 — Set Member as Recipient

Type: Set node **What it does:** Sets `recipientEmail = memberEmail` and `recipientName = memberName`. **Why:** Default path — member receives the quote directly.

Node 6 — Merge

Type: Merge node **What it does:** Combines both IFA and Member paths back into one flow. **Why:** After routing, both paths need to continue to the same Download and Gmail nodes. The merge brings them back together.

Node 7 — Download Letter from Drive

Type: Google Drive node **What it does:** Downloads the file identified by `documentFileId` from Google Drive as binary data. **Why:** Gmail requires the actual file content to attach it — not just the Drive link. This downloads the binary so it can be attached to the email.

Node 8 — Send Gmail with Attachment

Type: Gmail node **What it does:** Sends email to `recipientEmail` with:

- Subject: "Your Retirement Quote — Case {bestCaseId}"
- Body: personalised message with member name, scheme name, case reference

- Attachment: the downloaded letter binary **Why:** This is the delivery mechanism for the retirement quote. Replaces the RPA BOT which did the same thing via UI automation. Gmail node is more reliable, faster, and easier to maintain.

Node 9 — Gmail Sent?

Type: IF node **What it does:** Checks if Gmail returned a message `id` (confirms the email was accepted and queued for delivery). **Why:** Gmail API can fail (auth issues, quota exceeded, etc). Must confirm successful send before updating CAP and BeSt.

- **True** → Update CAP - Issued
- **False** → ERROR - Gmail Failed

Node 10 — ERROR - Gmail Failed

Type: Set node **What it does:** Returns `rpaStatus: "FAILED"` with error details and `alertAdmin: true`. **Why:** Failed email delivery means the member never received their quote. This must be flagged immediately for manual follow-up.

Node 11 — Update CAP - Issued

Type: HTTP Request node **What it does:** `PUT http://127.0.0.1:5005/api/cases/{bestCaseId}/communication` with `{ status: "ISSUED", issuedAt, commType: "email", sentTo: recipientEmail }`. **Why:** CAP must know the communication was sent so it can update the case status and stop any pending reminder tasks.

Node 12 — Log Issued

Type: Set node **What it does:** Sets `communicationStatus: "ISSUED"`, `rpaStatus: "COMPLETED"`, `botStatus: "COMPLETED"` with timestamp. **Why:** Records the communication event with full details for auditing.

Node 13 — Push to Final Review

Type: HTTP Request node **What it does:** `POST http://127.0.0.1:5005/api/cases/{bestCaseId}/final-review` with `{ queue: "ADMIN_REVIEW", pushedAt, reason: "Retirement quote issued — pending final review",`

`sentTo: recipientEmail }`. **Why:** After the quote is issued, the case doesn't close automatically. It enters a Final Review queue where an admin checker reviews all steps were completed correctly, the checklist is signed off, and the case can be officially closed. This is a compliance requirement.

Node 14 — Edit Fields

Type: Set node **What it does:** Final terminal output with complete envelope: `status: "SUCCESS"`, all communication fields, all member fields from Prepare Payload, `finalReviewPushed: true`. **Why:** WF01 reads this to confirm communication completed. All member fields must be present for the final Log Completion and Respond to CAP nodes.

---

10. WF07 — Validate Process Logic Sub-Workflow

Purpose: Calculates retirement type, pack type, and applicable scheme rules. Returns validated process parameters to WF01.

Total nodes: 13

Node 1 — Validate Process Entry

Type: Execute Workflow Trigger **What it does:** Receives all member data from WF01.

Node 2 — GET Member NRD from Supabase

Type: Supabase node **What it does:** Queries `member_nrd` table using `member_id = memberId`. Returns the member's Normal Retirement Date (NRD). **Why:** The retirement type (Early/Normal/Late) is determined by comparing the requested `retirementDate` against the NRD. Without fetching the NRD from the scheme data, this comparison is impossible.

Node 3 — Calculate Retirement Type and Pack Type

Type: Code node **What it does:**

- Compares `retirementDate` to NRD:
  - Before NRD → `retirementType = "Early"`
  - Same date → `retirementType = "Normal"`
  - After NRD → `retirementType = "Late"`

- Calculates months to retirement:
  - < 6 months → `packType = "FullPack"` (full quotation document)
  - ⩾ 6 months → `packType = "EstimateLetter"` (estimate only) **Why:** These two fields control what document WF04 produces and what BeSt records in the Ret Options section. The business rule (6 months = FullPack vs EstimateLetter) is a regulatory requirement.

Node 4 — GET DSS Scheme Config from Supabase

Type: Supabase node **What it does:** Queries `dss_scheme_config` table for `scheme_name = schemeName`. Returns: `early_retirement_allowed`, `trustee_approval_required`, `consent_required`, `scheme_nrd_age`, `automation_enabled`, `doc_prod_panel_details`, `pack_type`. **Why:** Each scheme has different rules. EGP may allow early retirement, JLR may require trustee approval — these rules are scheme-specific and stored in Supabase by the operations team.

Node 5 — Map DSS Config Fields

Type: Code node **What it does:** Maps Supabase column names (snake_case) to camelCase names used throughout the workflows. E.g. `early_retirement_allowed` → `earlyRetirementAllowed`. **Why:** n8n expressions and downstream workflows use camelCase. Supabase returns snake_case. This translation node prevents field name mismatches.

Node 6 — Retirement Type = Early?

Type: IF node **What it does:** Checks if `retirementType === "Early"`. **Why:** Early retirement has additional rule checks that Normal and Late don't need. Only Early retirement needs to be checked against scheme allowances.

- **True** → check if scheme allows early retirement
- **False** → Trustee Approval check

Node 7 — Early Retirement Allowed by Scheme?

Type: IF node **What it does:** Checks if `earlyRetirementAllowed === true` from the scheme config. **Why:** Some schemes simply do not allow members to retire before their NRD. If a member requests early retirement on such a scheme, the case must be closed — not processed.

- **True** → continue to Trustee Approval check
- **False** → RESULT - Early Retirement Not Allowed

Node 8 — RESULT - Early Retirement Not Allowed

Type: Set node **What it does:** Returns `validateProcessResult: "ONE_AND_DONE"` with reason `EARLY_RETIREMENT_NOT_ALLOWED`. **Why:** Case must be closed. WF01 routes ONE_AND_DONE to WF05 which notifies the member.

Node 9 — Trustee Approval Required?

Type: IF node **What it does:** Checks if `trusteeApprovalRequired === true`. **Why:** Some schemes require the pension trustee board to approve early retirements. This flag must be set so the admin team knows approval must be obtained before the quote can be issued.

- **True** → Flag - Trustee Approval Needed
- **False** → Consent Required check

Node 10 — Flag - Trustee Approval Needed

Type: Set node **What it does:** Sets `preApprovalsRequired: "true"` and `preApprovalType: "TRUSTEE_APPROVAL"` with an explanatory message. **Why:** Flags the requirement without stopping the automation — the system records the need for approval and continues. The checker at Final Review ensures approval is obtained before close.

Node 11 — Consent Required?

Type: IF node **What it does:** Checks if `consentRequired === true`. **Why:** Some schemes require the member's explicit written consent before a retirement quote can be issued. This flag ensures it's noted.

- **True** → Flag - Consent Needed
- **False** → Compile Validated Process Parameters

Node 12 — Flag - Consent Needed

Type: Set node **What it does:** Sets `preApprovalsRequired: "true"` and `preApprovalType:`

`"CONSENT_REQUIRED"`. **Why:** Records consent requirement. Checker reviews at Final Review.

Node 13 — Compile Validated Process Parameters

Type: Set node **What it does:** Final output. Returns complete validated parameters: `validateProcessResult: "PASS"`, `retirementType`, `packType`, `preApprovalsRequired`, `preApprovalType`, `trusteeApprovalRequired`, `consentRequired`, `schemeRetirementAge`, plus all member fields. **Why:** WF01 needs `retirementType` and `packType` to pass to WF03 (Ret Options) and WF04 (letter content). All member fields must be re-output as the Supabase nodes replaced `$json`.

---

11. WF08 — Split POS Cases Sub-Workflow

Purpose: Handles members with multiple Periods of Service. Creates one BeSt case per POS and returns ready-to-process case data.

Total nodes: 8

Node 1 — Split POS Entry

Type: Execute Workflow Trigger **What it does:** Receives all member data from WF01 including `posCount` and `posDetails`.

Node 2 — GET Full POS Details from BeSt

Type: HTTP Request node **What it does:** `GET http://127.0.0.1:5005/api/members/{memberId}/periods-of-service?scheme={schemeName}&includeDetails=true`. Returns full POS details from mock server. **Why:** WF02 flags that multiple POS exist but doesn't fetch full details. WF08 fetches the complete POS data including start dates, end dates, benefit names, and member multipart IDs needed to create separate BeSt cases.

Node 3 — GET Scheme Membership Details

Type: HTTP Request node **What it does:** `GET http://127.0.0.1:5005/api/scheme-membership/{memberId}?scheme={schemeName}`. Returns scheme membership details. **Why:** Each period of service may have different membership details within the same scheme. These are needed to correctly configure each individual BeSt case.

Node 4 — POS Count > 1?

Type: IF node **What it does:** Checks if `posCount > 1`. **Why:** Even though WF02 flagged multiple POS, this confirmation check uses the actual API response data. If the payload said posCount=2 but the API returns only 1 POS, we handle it correctly.

- **True** → Create One Item Per POS
- **False** → Single POS - No Split Needed

Node 5 — Single POS - No Split Needed

Type: Set node **What it does:** Returns a single POS record with `posId: "DEFAULT"`, `splitCase: false`, and all member fields. `posReady: true`. **Why:** Most members have one period of service. This path handles them cleanly without the splitting logic — passes through with default POS fields set correctly.

Node 6 — Create One Item Per POS

Type: Code node **What it does:** Takes the `posDetails` array and creates one output item per POS. Each item contains: `posIndex`, `totalPosCount`, `posId`, `posStartDate`, `posEndDate`, `benefitName`, `memberMultipartId`, `splitCase: true`, and a unique `bestCaseId` suffixed with `_POS{index}`. Also spreads all original member fields. **Why:** n8n processes each output item separately. By creating multiple items here, the next node (Create BeSt Case per POS) runs once per item — automatically creating one BeSt case per period of service. This is the key mechanism for POS splitting.

Node 7 — Create BeSt Case per POS

Type: HTTP Request node **What it does:** `POST http://127.0.0.1:5005/api/cases` for each POS item. Body includes `memberId`, `schemeName`, `processType`, `posId`, `posIndex`, `benefitName`, `retirementDate`, `parentCaseId`. **Why:** Each Period of Service needs its own BeSt case. This POST creates them one by one (n8n loops through each item from the code node). In production, this registers separate trackable cases in BeSt for each employment period.

Node 8 — POS Case Ready

Type: Set node **What it does:** Final output for each POS. Sets `posReady: true`, `posIndex`,

`totalPosCount`, `bestCaseId` (the new per-POS case ID), `posId`, `benefitName`, `splitCase`, plus all member fields from Split POS Entry. **Why:** WF01 receives this output (one per POS) and continues the full automation for each period. The `bestCaseId` is now the POS-specific case ID which all subsequent BeSt stage updates reference.

---

12. The Payload — Every Field Explained

json


```
{
  "bestCaseId": "CASE-2024-001",
  "memberId": "MBR-AB123456C",
  "schemeName": "EGP",
  "schemeCode": "EGP001",
  "processType": "Retirement Quote",
  "memberName": "James Richardson",
  "memberEmail": "james.richardson@email.com",
  "dateOfBirth": "1967-04-15",
  "currentAddress": "12 Oak Lane, Manchester, M1 2AB",
  "retirementDate": "2025-06-01",
  "membershipStatus": "Active",
  "commPreference": "email",
  "posCount": 1,
  "hasAVC": false,
  "senderType": "member",
  "ifaEmail": "",
  "ifaName": "",
  "illHealth": false,
  "vulnerableFlag": false
}
```

Field	What It Is	What Controls It
`bestCaseId`	Unique case reference — created in BeSt	All HTTP calls, email subject line, Drive file names
`memberId`	Unique member ID	DPA Supabase lookup, mock server calls, Drive folder name
`schemeName`	Which pension scheme (EGP, Cadent, JLR etc)	Scope check, DSS Supabase lookup, letter content
`schemeCode`	Short alphanumeric code for the scheme	Passed through to BeSt and documents
`processType`	Must be "Retirement Quote"	Gate in WF01 — wrong type stops everything
`memberName`	Full name of the member	Email salutation, letter header
`memberEmail`	Member's email address	Email recipient when senderType=member
`dateOfBirth`	Date of birth (YYYY-MM-DD)	Age calculation — must be ⩾55 at retirementDate
`currentAddress`	Home address	Letter content — included in correspondence
`retirementDate`	Requested retirement date	Age check, retirement type vs NRD, pack type calc
`membershipStatus`	Active / Deferred / No Liability / Pensioner / Other	Routes case to automation, ONE_AND_DONE, or ADMIN_CASE
`commPreference`	How member wants to be contacted	Currently only "email" supported
`posCount`	Number of periods of service	1 = single flow; 2+ = WF08 splits into multiple cases
`hasAVC`	Additional Voluntary Contributions ticked	true = pause automation, email admin, wait for AVC value
`senderType`	"member" or "IFA"	Controls who receives the email in WF06
`ifaEmail`	IFA's email address	Email recipient when senderType=IFA

Field	What It Is	What Controls It
`ifaName`	IFA's full name	Email salutation when sending to IFA
`illHealth`	Ill health retirement request	true = ONE_AND_DONE — needs specialist team
`vulnerableFlag`	Member is flagged as vulnerable	true = full flow continues but flagged for checker review

13. All Test Scenarios

Scenario	Change From Default	Expected Result
✅ Happy Path	All defaults	Full flow → 3 files in Drive → email sent → ISSUE_DOCUMENTATION updated → Final Review pushed
🔴 Wrong Scheme	`"schemeName": "UNKNOWN_SCHEME"`	Stops at Node 3 — STOP Not In Scope
🔴 Wrong Process	`"processType": "Transfer Out"`	Stops at Node 5 — STOP Wrong Process
🔴 Under 55	`"dateOfBirth": "1995-04-15"`	WF02 CHECK 1 fails → WF05 closes case — Under 55 message
🔴 DPA Failed	`"memberId": "MBR-UNKNOWN-999"`	WF02 CHECK 2 fails — not in Supabase → WF05 closes — DPA Failed message
🔴 No Liability	`"membershipStatus": "No Liability"`	WF02 CHECK 3 → WF05 closes — No Liability message
🔴 Pensioner	`"membershipStatus": "Pensioner"`	WF02 CHECK 3 → WF05 closes — Pensioner message
🔴 Ill Health	`"illHealth": true`	WF02 CHECK 6 → WF05 closes — Ill Health message
⚠️ Vulnerable	`"vulnerableFlag": true`	Full flow continues — `vulnerableFlag: true` carried through to final output

Scenario	Change From Default	Expected Result
⏸️ AVC Hold	`"hasAVC": true`	Admin receives email with form link → workflow pauses → admin submits AVC value → continues
📋 Admin Case	`"membershipStatus": "Unknown"`	WF02 RESULT - Other Status → ADMIN_CASE → Create Admin Team Case HTTP call
📨 IFA Routing	`"senderType": "IFA", "ifaEmail": "advisor@firm.com", "ifaName": "Robert Clarke"`	Full flow → email sent to IFA not member
🔀 Multi POS	`"posCount": 2`	WF08 creates 2 BeSt cases — automation runs for each POS

14. Key Things To Tell The Client

"This is production-ready logic in a demo environment." Every routing decision, every eligibility check, every BeSt stage update — all of this reflects exactly what would happen in production. The only differences are:

- Google Drive instead of SharePoint (same upload/download pattern)
- Gmail instead of RPA BOT (same email delivery outcome)
- Text files instead of DocProd PDFs (same file → Drive → email flow)
- Mock server instead of real CAP/BeSt APIs (same endpoints, same data structure)

"The architecture is designed for real system integration." Swapping in the real DocProd API, SharePoint, and BeSt APIs requires only changing the HTTP node URLs and credentials — the entire workflow logic stays exactly the same.

"Human in the loop is built and working." The AVC Wait node genuinely pauses execution. The form is real. When the admin submits, the workflow resumes automatically. This isn't simulated.

"Every case has a full audit trail." Three files in Drive per case. Four BeSt stage updates. Milestone questions updated. CAP communication record. Final Review push. Every step is logged and traceable.